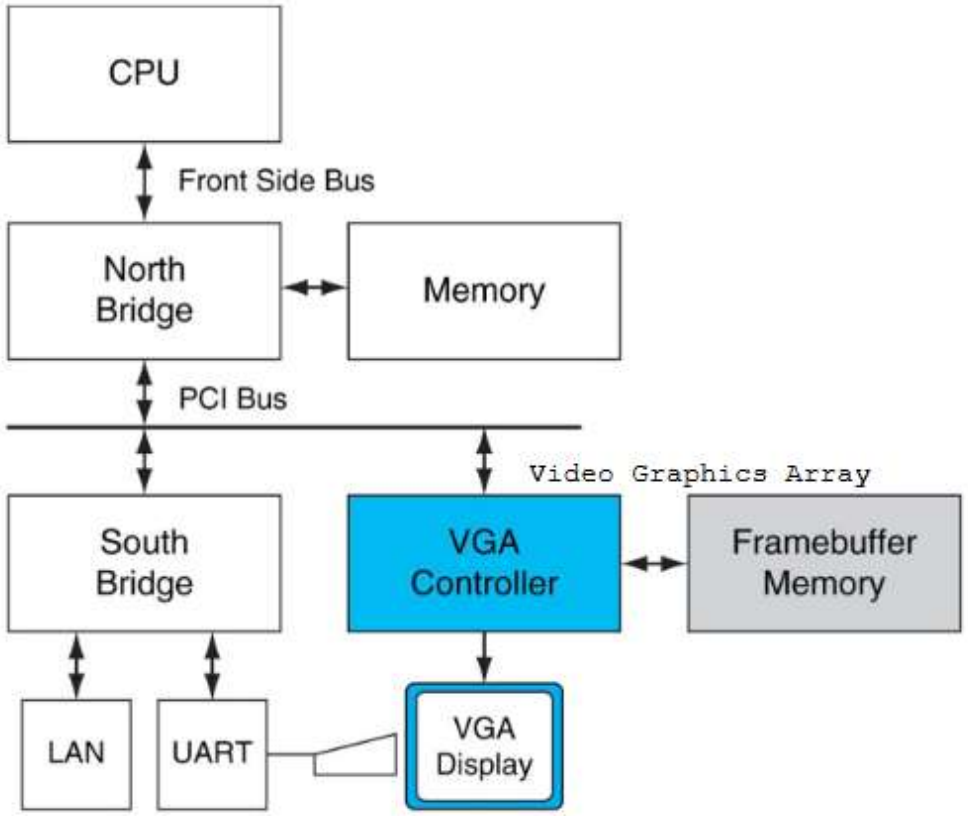
Lecture # 10

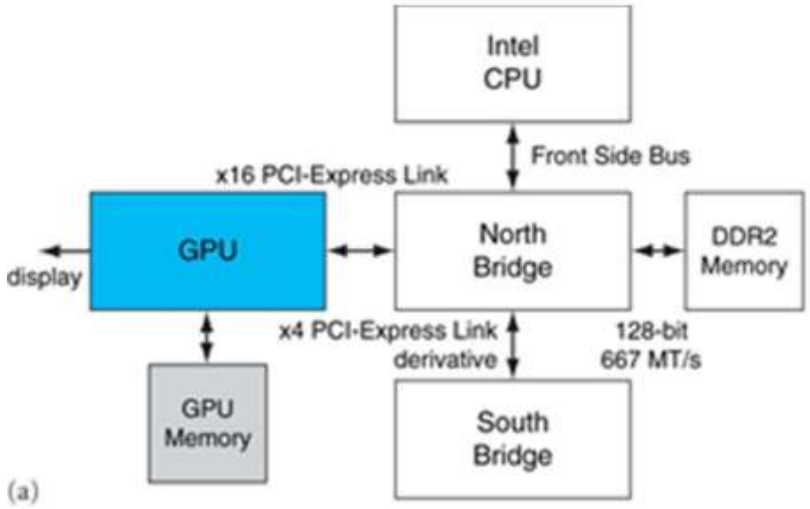
By: Muhammad Zain Uddin

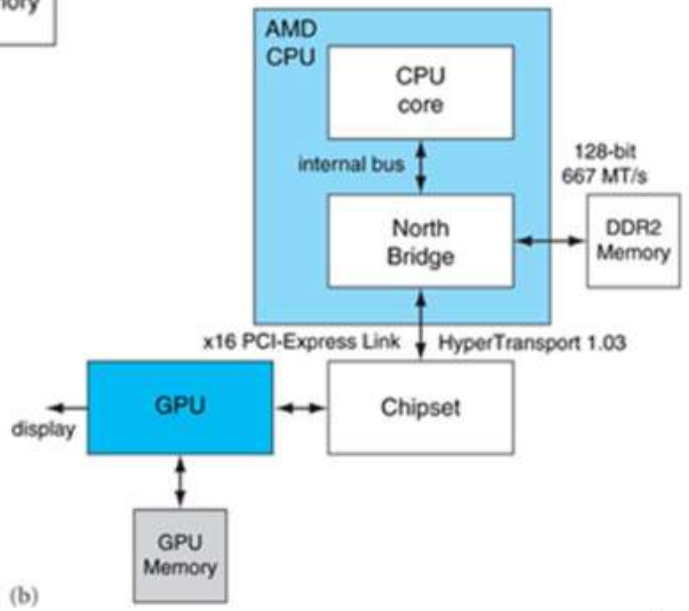
email: zuddin@iba.edu.pk

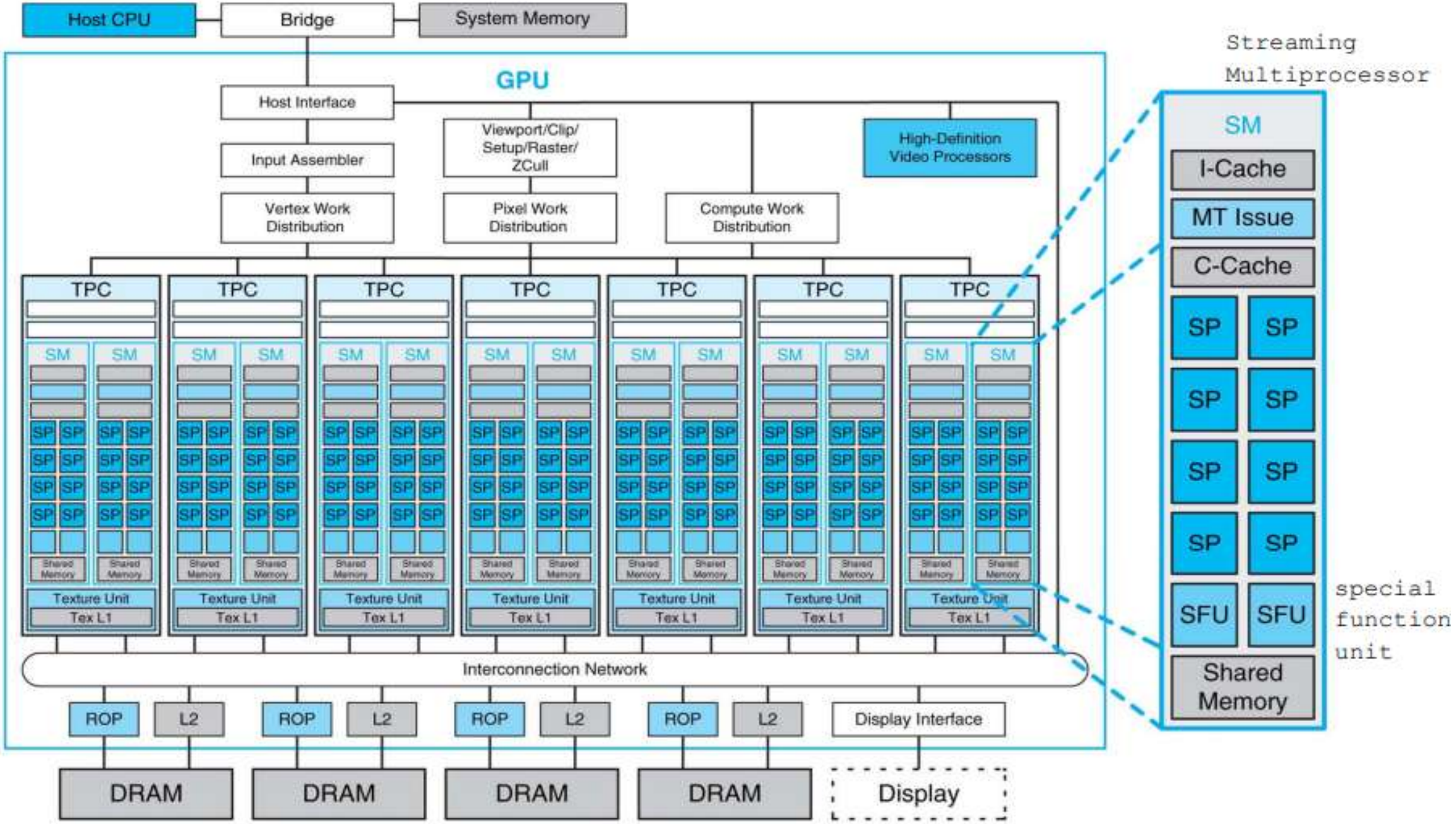
System with simple CPU and Video Cards



System with GPU






Host CPU
Bridge
System Memory
GPU
Host Interface
Input Assembler
Vertex Work Distribution
Viewport/Clip/ Setup/Raster/ ZCull
Pixel Work Distribution
Compute Work Distribution
High-Definition Video Processors
TPC
SM
SP
Shared Memory
Texture Unit
Tex L1
Interconnection Network
ROP
L2
Display Interface
DRAM
Display
Streaming Multiprocessor
SM
I-Cache
MT Issue
C-Cache
SP
SFU
Shared Memory
special function unit

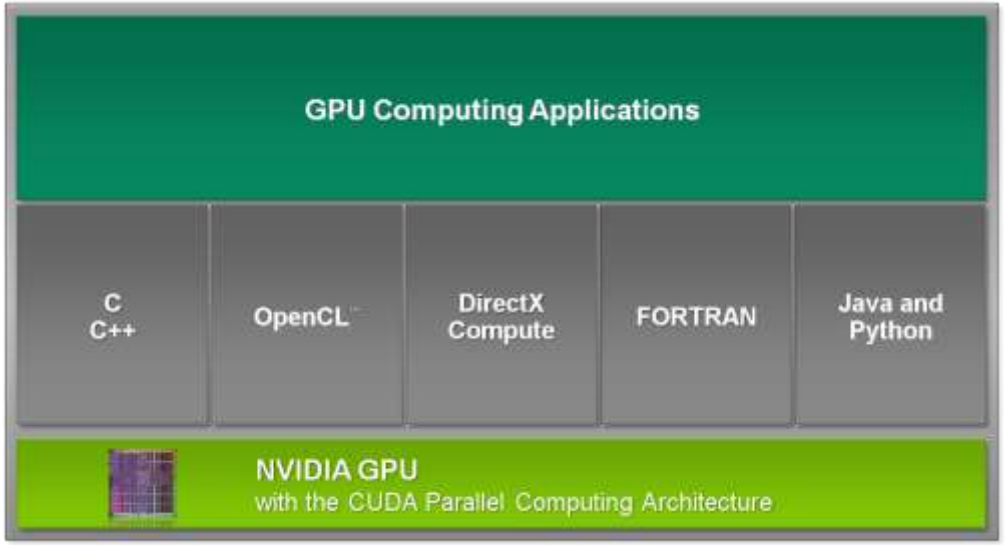
Various GPU Programming Models

- Overview of different GPU programming models:
  - CUDA (Compute Unified Device Architecture): Developed by NVIDIA for NVIDIA GPUs.
  - OpenCL (Open Computing Language): Cross-platform framework for parallel programming.
  - Vulkan Compute: Graphics and compute API for high-performance 3D graphics and parallel computation.
  - SYCL (Standard C++ for Parallelism): Standardized programming model for heterogeneous computing.
  - HIP (Heterogeneous-Compute Interface for Portability): Portable GPU kernel language developed by AMD.
  - ROCm (Radeon Open Compute Platform): Open-source software platform for GPU-accelerated computing on AMD GPUs.

Enter CUDA

CUDA is NVIDIA's general-purpose parallel computing architecture.

- designed for calculation-intensive computation on GPU hardware
- CUDA is not a language, it is an API
- we will mostly concentrate on the C implementation of CUDA



GPU Computing Applications

Libraries and Middleware

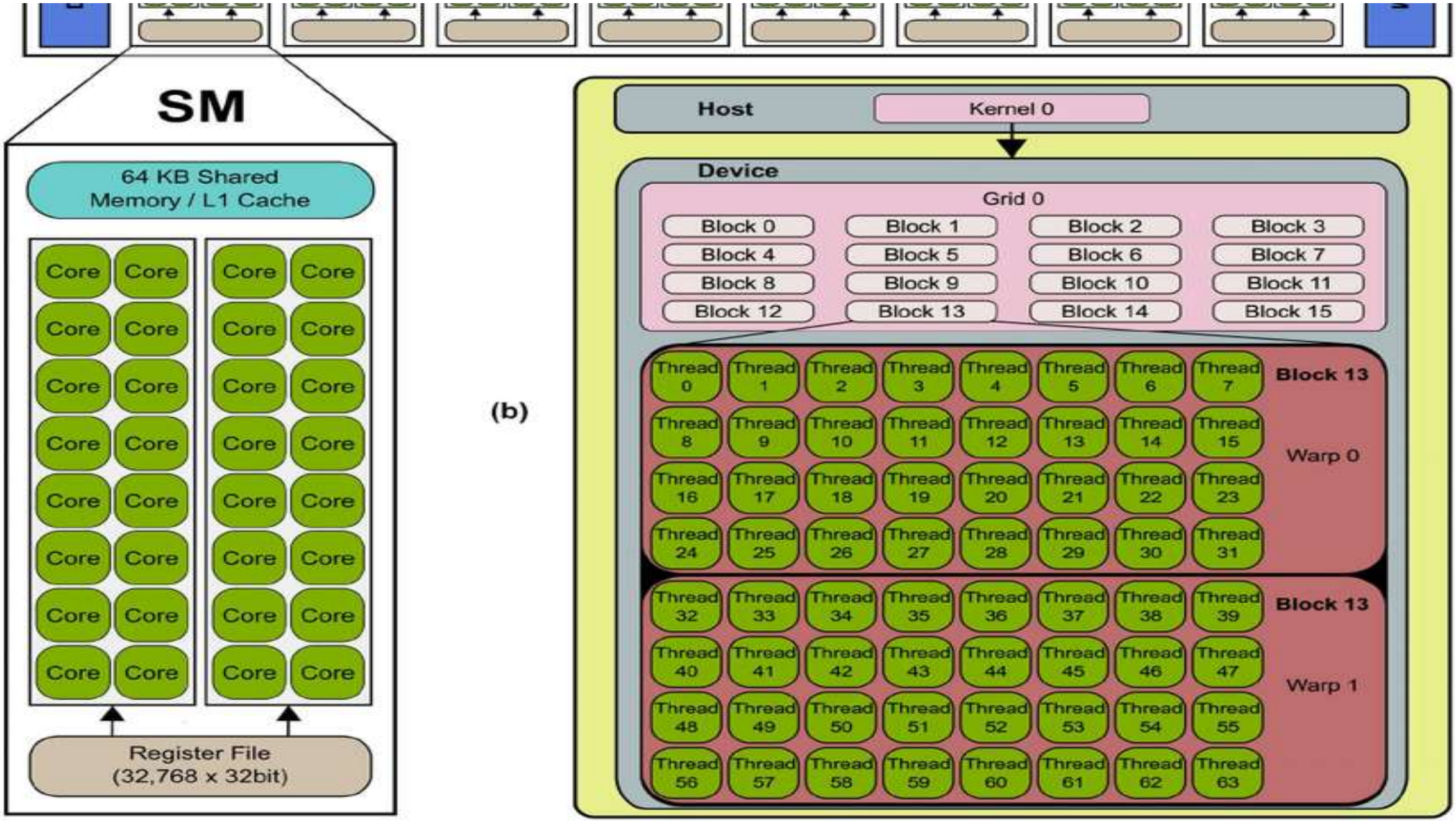
cuDNN TensorRT	cuFFT cuBLAS cuRAND cuSPARSE	CULA MAGMA	Thrust NPP	VSIPL SVM OpenCurrent	PhysX OptiX iRay	MATLAB Mathematica

Programming Languages

C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)

CUDA-Enabled NVIDIA GPUs

NVIDIA Ampere Architecture (compute capabilities 8.x)				Tesla A Series
NVIDIA Turing Architecture (compute capabilities 7.x)		GeForce 2000 Series	Quadro RTX Series	Tesla T Series
NVIDIA Volta Architecture (compute capabilities 7.x)	DRIVE/JETSON AGX Xavier		Quadro GV Series	Tesla V Series
NVIDIA Pascal Architecture (compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series	Tesla P Series
	Embedded	Consumer Desktop/Laptop	Professional Workstation	Data Center


SM
64 KB Shared Memory / L1 Cache
Core
Register File (32,768 x 32bit)
(b)
Host
Kernel 0
Device
Grid 0
Block 0
Block 1
Block 2
Block 3
Block 4
Block 5
Block 6
Block 7
Block 8
Block 9
Block 10
Block 11
Block 12
Block 13
Block 14
Block 15
Thread 0
Thread 1
Thread 2
Thread 3
Thread 4
Thread 5
Thread 6
Thread 7
Thread 8
Thread 9
Thread 10
Thread 11
Thread 12
Thread 13
Thread 14
Thread 15
Thread 16
Thread 17
Thread 18
Thread 19
Thread 20
Thread 21
Thread 22
Thread 23
Thread 24
Thread 25
Thread 26
Thread 27
Thread 28
Thread 29
Thread 30
Thread 31
Block 13
Warp 0
Thread 32
Thread 33
Thread 34
Thread 35
Thread 36
Thread 37
Thread 38
Thread 39
Thread 40
Thread 41
Thread 42
Thread 43
Thread 44
Thread 45
Thread 46
Thread 47
Thread 48
Thread 49
Thread 50
Thread 51
Thread 52
Thread 53
Thread 54
Thread 55
Thread 56
Thread 57
Thread 58
Thread 59
Thread 60
Thread 61
Thread 62
Thread 63
Block 13
Warp 1

Kernel Function Qualifiers

__device__

__global__

__host__

Example in C:

CPU program

```
void increment_cpu(float *a, float b, int N)
```

CUDA program

```
__global__ void increment_gpu(float *a, float b, int N)
```

Variable Type Qualifiers

Specify how a variable is stored in memory

__device__

__shared__

__constant__

Example:

```
__global__ void increment_gpu(float *a, float b, int N)
{
 __shared__ float shared[];
}
```

Calling the Kernel

Calling a kernel function is much different from calling a regular function

```
Void main(){
Int blocks = 256;
Int threadsperblock = 512;
 mycudafunc<<<blocks,threadsperblock>>>(some parameter);
}
```

GPU Memory Allocation / Release

Host (CPU) manages GPU memory:

```
 cudaMalloc (void ** pointer, size_t nbytes)
 cudaMemset (void * pointer, int value, size_t count);
 cudaFree (void* pointer)
Void main(){
 int n = 1024;
 int nbytes = 1024*sizeof(int);
 int * d_a = 0;
 cudaMalloc( (void**)&d_a,  nbytes );
 cudaMemset( d_a, 0, nbytes);
 cudaFree(d_a);
}
```

Memory Transfer

cudaMemcpy(void *dst, void *src, size_t nbytes, enum cudaMemcpyKind direction);

returns after the copy is complete blocks CPU

thread doesn’t start copying until previous CUDA calls complete

enum cudaMemcpyKind

cudaMemcpyHostToDevice

cudaMemcpyDeviceToHost

cudaMemcpyDeviceToDevice

Host Synchronization

All kernel launches are asynchronous

- control returns to CPU immediately
- kernel starts executing once all previous CUDA calls have completed

Memcopies are synchronous

- control returns to CPU once the copy is complete
- copy starts once all previous CUDA calls have completed

cudaThreadSynchronize()

- blocks until all previous CUDA calls complete

Asynchronous CUDA calls provide:

- non-blocking memcopies
- ability to overlap memcopies and kernel execution

The Big Difference

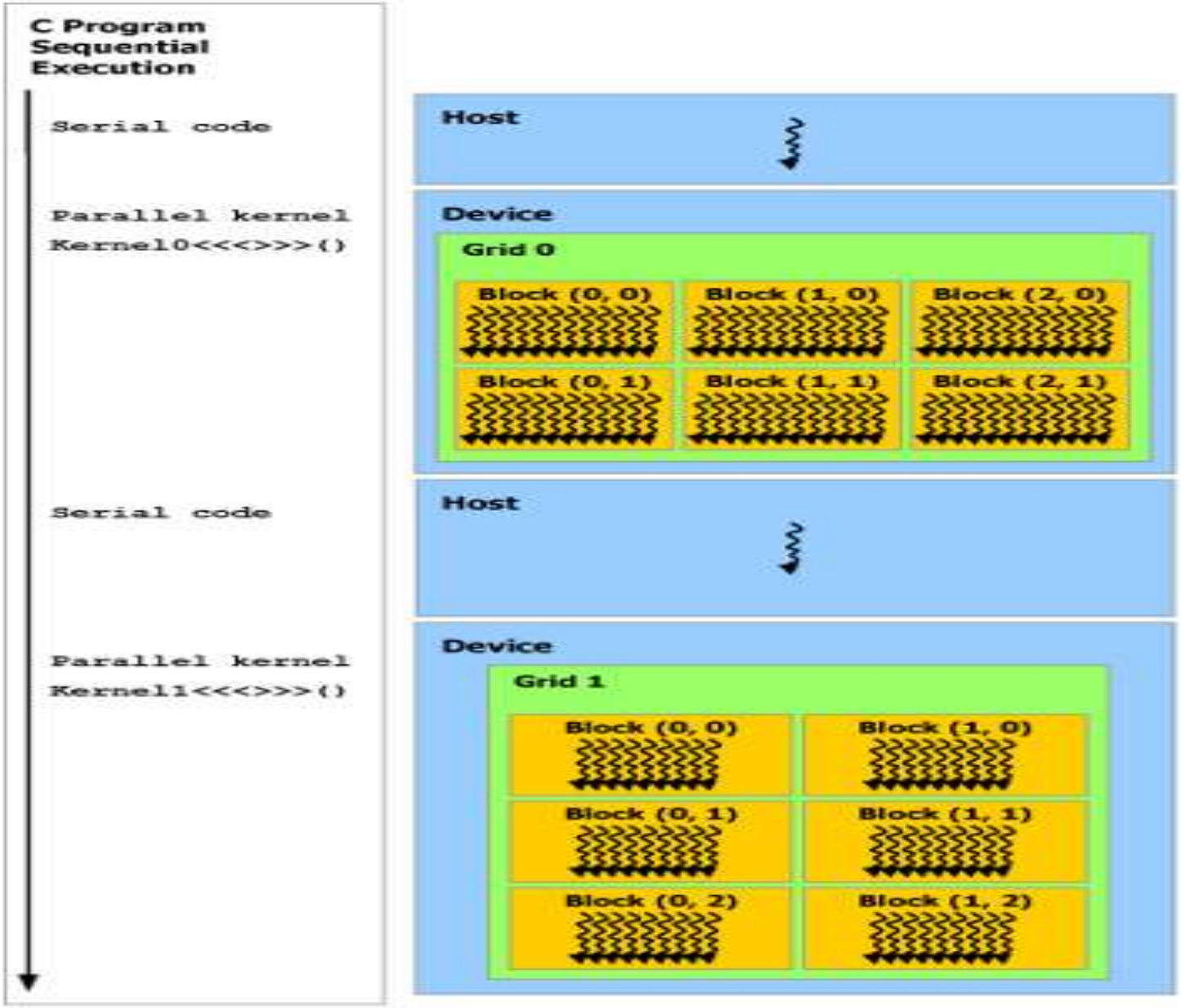
CPU PROGRAM

```
void increment_cpu(float *a, float b, int N)
{
 for (int idx = 0; idx<N; idx++)
a[idx] = a[idx] + b;
}

void main()
{
…
increment_cpu(a, b, N);
}
```

GPU PROGRAM

```
__global__ void increment_gpu(float *a, float b, int N)
 {
int idx = blockIdx.x * blockDim.x + threadIdx.x;        if( idx
  < N)                a[idx] = a[idx] + b;
}
void main() {
…..
dim3 dimBlock (blocksize);
 dim3 dimGrid( ceil( N / (float)blocksize)  )
  increment_gpu<<<dimGrid, dimBlock>>>(a, b, N);
}
```


C Program
Sequential
Execution
Serial code
Parallel kernel
Kernel0<<<>>>()
Serial code
Parallel kernel
Kernel1<<<>>>()
Host
Device
Grid 0
Block (0, 0)
Block (1, 0)
Block (2, 0)
Block (0, 1)
Block (1, 1)
Block (2, 1)
Host
Device
Grid 1
Block (0, 0)
Block (1, 0)
Block (0, 1)
Block (1, 1)
Block (0, 2)
Block (1, 2)

CUDA C Examples

Vector Addition

https://github.com/olcf-tutorials/vector_addition_cuda

Finding Maximum Value in an array

https://www.geeksforgeeks.org/how-to-run-cuda-c-c-on-jupyter-notebook-in-google-colaboratory/

Matrix Addition

https://github.com/jcbacong/CUDA-matrix-addition

Matrix Multiplication

https://github.com/lzhengchun/matrix-cuda

GPU Compute Capability and Limits

Technical Specifications per Compute Capability from 5.0 to 9.0

Maximum dimensionality of grid of thread blocks	3
Maximum x -dimension of a grid of thread blocks	2^{31}-1
Maximum y- or z-dimension of a grid of thread blocks	65535
Maximum dimensionality of thread block	3
Maximum x- or y-dimensionality of a block	1024
Maximum z-dimension of a block	64
Maximum number of threads per block	**1024**

https://developer.nvidia.com/cuda-gpus

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#features-and-technical-specifications

Reference

Book : CUDA C/C++ Programming Guide

https://docs.nvidia.com/cuda/cuda-c-programming-guide

Book : PROFESSIONAL CUDA® C Programming – 2014

https://www.cs.utexas.edu/~rossbach/cs380p/papers/cuda-programming.pdf

Tutorials (CMU &

https://people.cs.pitt.edu/~melhem/courses/xx45p/cuda1.pdf

https://www.cs.cmu.edu/afs/cs/academic/class/15418-s18/www/lectures/06_gpuarch.pdf

Workshop (Cornell)

https://cvw.cac.cornell.edu/GPUarch/default

GPU Programming

NVIDIA - CUDA SETUP FOR GOOGLE COLAB

GPU Programming

CUDA setup For Google CoLab

- CUDA C/C++
- CUDA Python

Example Programs in CUDA-C & CUDA-Python

CUDA-C/C++ setup For Google CoLab

Step 1: remove old setup (If you have old setup)

```
!apt-get --purge remove cuda nvidia* libnvidia-*
!dpkg -l | grep cuda- | awk '{print $2}' | xargs -n1 dpkg --purge
!apt-get remove cuda-*
!apt autoremove
!apt-get update
```

CUDA-C/C++ setup For Google CoLab

Step 2: install latest cuda

1. **Checking ubuntu version on colab :**

```
!lsb_release -a
```

2. **Getting setup code from nvidia koolkit site for Ubuntu 20.04.5 LTS**
   https://developer.nvidia.com/cuda-downloads

   or search for specific version

CUDA-C/C++ setup For Google CoLab

Step 2: install latest cuda (If latest is not installed)

3. Run the following commands obtained from the official site (for cuda 11.3)

```
!wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/x86_64/cuda-ubuntu2004.pin
!sudo mv cuda-ubuntu2004.pin /etc/apt/preferences.d/cuda-repository-pin-600
!wget https://developer.download.nvidia.com/compute/cuda/11.3.0/local_installers/cuda-repo-ubuntu2004-11-3-local_11.3.0-465.19.01-1_amd64.deb
!sudo dpkg -i cuda-repo-ubuntu2004-11-3-local_11.3.0-465.19.01-1_amd64.deb
!sudo apt-key add /var/cuda-repo-ubuntu2004-11-3-local/7fa2af80.pub
!sudo apt-get update
!sudo apt-get -y install cuda
```

Or you can use the following link if you have the different machine or OS
[CUDA Toolkit 11.3 Downloads | NVIDIA Developer]

CUDA-C/C++ setup For Google CoLab

Step 3: check Cuda version

```
!nvcc --version
```

Step 4: extension to run nvcc from the Notebook cells.

```
!pip install git+https://github.com/andreinechaev/nvcc4jupyter.git
```

Step 5:load the extension

```
%load_ext nvcc4jupyter
```

CUDA-C/C++ setup For Google CoLab

Step 6: test run code

```
%%cuda
#include <stdio.h>

__global__ void hello(){
    printf("Hello from block: %u, thread: %u\n", blockIdx.x, threadIdx.x);
}

int main(){
    hello<<<2, 2>>>();
    cudaDeviceSynchronize();
}
```

CUDA-C Example Programs

Vector Addition in CUDA-C

https://github.com/olcf-tutorials/vector_addition_cuda

Finding maximum value in an array

How To Run CUDA C/C++ on Jupyter notebook in Google Colaboratory

https://www.geeksforgeeks.org/how-to-run-cuda-c-c-on-jupyter-notebook-in-google-colaboratory/

CUDA-Python Example Programs

Tutorial : Introduction to CUDA and PyCUDA

https://colab.research.google.com/github/vellamike/pycuda/blob/master/pycuda_workshop.ipynb#scrollTo=FA_YN7HlGRP5

Five different ways to sum vectors in PyCUDA

https://vitalitylearning.medium.com/five-different-ways-to-sum-vectors-in-pycuda-3f2d9409b139

Attribution

TODAY'S LECTURE IS BASED ON BEEJ'S GUIDE TO INTER-PROCESS COMMUNICATION

fork()

fork() is how Unix starts new processes

Basically, how it works is this:

- the parent process (the one that already exists) fork()'s a child process (the new one)
- The child process gets a copy of the parent's data.

Something to note about process behavior under Unix:

- When a process dies, it doesn't really go away completely.
- It's no longer running, but a small remnant is waiting around for the parent process to pick up.
- So after a parent process fork()s a child process, it must wait() for that child process to exit.
- It is this act of waiting that allows all remnants of the child to vanish.

fork()

There is an exception to the above rule: the parent can ignore the SIGCHLD signal and then it won’t have to wait()

signal(SIGCHLD, SIG_IGN); /* now I don't have to wait()! */

When a child process dies and has not been wait()ed on, it will usually show up in a ps listing as “<defunct>”.

When the parent dies before it wait()s for the child (assuming it is not ignoring SIGCHLD), the child is reparented to the init process (PID 1).

- This is not a problem if the child is still living well and under control.

fork()

However, if the child is already defunct, we're in a bit of a bind. See, the original parent can no longer wait(), since it's dead.

Thankfully, on some systems, init periodically destroys all the defunct processes it owns.

On other systems, it outright refuses to become the parent of any defunct processes, instead destroying them immediately.

fork()

To summarize:

Children become defunct until the parent wait()s, unless the parent is ignoring SIGCHLD.

Furthermore, children (living or defunct) whose parents die without waiting for them (again assuming the parent is not ignoring SIGCHLD) become children of the init process, which deals with them heavy handedly.

fork() - example

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
 pid_t pid;
 int rv;

 switch(pid = fork()) {
 case -1:
   perror("fork");
   exit(1);

 case 0:
   printf("CHILD: Enter my exit status      (make it small): ");
   scanf(" %d", &rv);
   exit(rv);

 default:
   printf("PARENT: I'm now waiting for my child to exit()...\n");
   wait(&rv);
   printf("PARENT: My child's exit status is: %d\n", WEXITSTATUS(rv));
   printf("PARENT: I'm outta here!\n");
 }

 return 0;
}
```

Question

DOES WAIT() KNOW WHICH CHILD PROCESS TO WAIT FOR?

wait()

By default, it waits for whichever one happens to exit first.

If you must, you can specify exactly which child to wait for by calling waitpid() with the child's PID as an argument

Question

BOTH PARENT AND CHILD USE THE RV VARIABLE. DOES THIS MEAN THAT IT IS SHARED BETWEEN THE PROCESSES?

Signals

There is a sometimes useful method for one process to bug another: signals.

Basically, one process can “raise” a signal and have it delivered to another process.

The destination process’s signal handler (just a function) is invoked and the process can handle it.

- What you are permitted to do safely inside your signal handler is rather limited.

Signals

For example, one process might want to stop another one, and this can be done by sending the signal SIGSTOP to that process. To continue, the process has to receive signal SIGCONT.

How does the process know to do this when it receives a certain signal?

Well, many signals are predefined and the process has a default signal handler to deal with it.

- Take SIGINT for example. This is the interrupt signal that a process receives when the user hits ^C. The default signal handler for SIGINT causes the process to exit!

Signal

SIGABRT Process abort signal.

SIGFPE Erroneous arithmetic operation.

SIGILL Illegal instruction.

SIGINT Terminal interrupt signal.

SIGKILL Kill (cannot be caught or ignored).

SIGPIPE Write on a pipe with no one to read it.

SIGTERM Termination signal.

SIGUSR1 User-defined signal 1.

SIGUSR2 User-defined signal 2.

SIGCHLD Child process terminated or stopped.

SIGCONT Continue executing, if stopped.

SIGSTOP Stop executing (cannot be caught or ignored).

SIGSYS Bad system call.

SIGTRAP Trace/breakpoint trap.

SIGXFSZ File size limit exceeded.

Question

HOW DO YOU CATCH A SIGNAL?

Catching Signals

You need to call sigaction() and tell it all the gritty details about which signal you want to catch and which function you want to call to handle it.

```
int sigaction(int sig, const struct sigaction *act, struct sigaction
*oact);
```

The first parameter, sig is which signal to catch. This can be (probably “should” be) a symbolic name from signal.h along the lines of SIGINT.

The next field, act is a pointer to a struct sigaction which has a bunch of fields that you can fill in to control the behavior of the signal handler.

Lastly oact can be NULL, but if not, it returns the old signal handler information that was in place before. This is useful if you want to restore the previous signal handler at a later time.

Signals - example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>

void sigint_handler(int sig)
{
    const char msg[] = "Ahhh! SIGINT!\n";
    write(0, msg, sizeof(msg));
}

int main(void)
{
    char s[200];
    struct sigaction sa = {
        .sa_handler = sigint_handler,
        .sa_flags = 0, // or SA_RESTART
        .sa_mask = 0,
    };
    if (sigaction(SIGINT, &sa, NULL) == -1) {
        perror("sigaction");
        exit(1);
    }

    printf("Enter a string:\n");

    if (fgets(s, sizeof s, stdin) == NULL)
        perror("fgets");
    else
        printf("You entered: %s\n", s);
    return 0;
}
```

Question

WILL THIS PROGRAM END UP TERMINATING SUCCESSFULLY SINCE WE HAVE MODIFIED THE SIGINT BEHAVIOUR?

Question

DO YOU SEE SOMETHING ODD IN THE PROGRAM?

(HINT: IT IS RELATED TO PRINTING TEXT ON SCREEN)

Signals - Caution

Some system calls can be interrupted by signals, and when this happens, they return an error

- In our example, fgets() returns an error, namely EINTR, or “Interrupted system call”.

You also have to be careful when you make function calls in your signal handler. Those functions must be “async safe”, so they can be called without invoking undefined behavior.

- POSIX says that write() is async-safe (so is safe to call from within the handler), while printf() is not.
- Of course, you can call your own functions from within your signal handler (as long they don’t call any non-async-safe functions.)

Signals - Caution

You also cannot safely alter any shared (e.g. global) data, with one notable exception: variables that are declared to be of storage class and type volatile sig_atomic_t.

Signal – another example

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <signal.h>

volatile sig_atomic_t got_usr1;

void sigusr1_handler(int sig)
{
    got_usr1 = 1;
}

 int main(void)
{
    struct sigaction sa = {
        .sa_handler = sigusr1_handler,
        .sa_flags = 0,
        .sa_mask = 0,
    };
    got_usr1 = 0;

 if (sigaction(SIGUSR1, &sa, NULL) == -1)
   {
        perror("sigaction");
        exit(1);
    }

    while (!got_usr1) {
        printf("PID %d:\n", getpid());
        sleep(1);
    }

    printf("Done in by SIGUSR1!\n");

    return 0;
}
```